

Week 9

Systems Security

IN 1011 Operating Systems

Systems Security

- Computer Security Overview
- Problems: Security Violation Methods
- Actions to Mitigate the Problems
- Authentication, Passwords and Alternatives
- More on Threats

Systems Security

- Computer Security Overview
- Problems: Security Violation Methods
- Actions to Mitigate the Problems
- Authentication, Passwords and Alternatives
- More on Threats

Protection $\neq$ Security

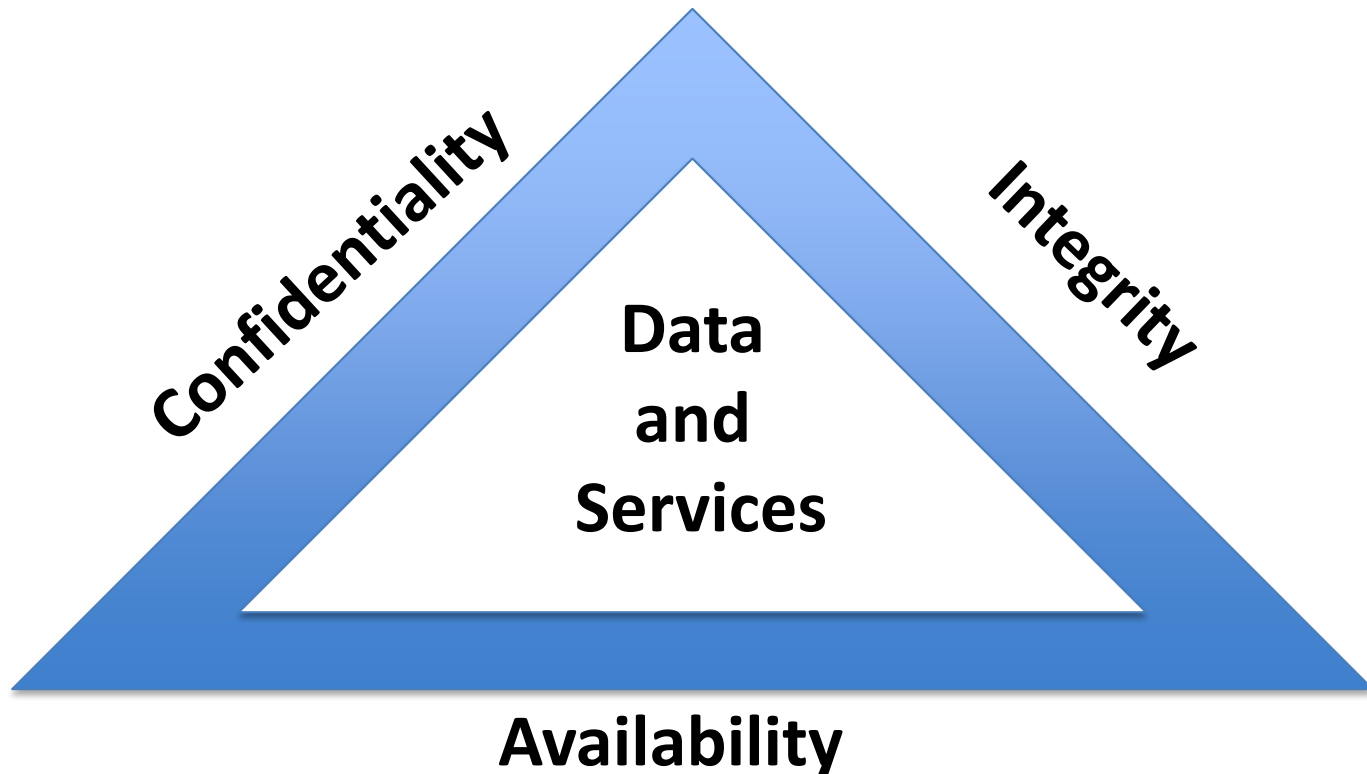
- File system protection mechanisms may prevent a process owned by watson from accessing a file owned by sherlock.
- But what if watson can login as sherlock (masquerading)?
 - Then watson can execute processes owned by sherlock
 - So watson can see and modify sherlock's data and processes

Computer Security Overview

- The National Institute of Standards and Technology (NIST <https://www.nist.gov/>) Computer Security Handbook 1995 (<https://www.nist.gov/publications/introduction-computer-security-nist-handbook>) defines the term Computer Security as:
 - “The protection afforded to an automated information system in order to attain the applicable objectives of preserving the **integrity**, **availability** and **confidentiality** of information system resources”

Computer Security Overview

- Preserving the **integrity**, **availability** and **confidentiality** is sometimes known as the **CIA Triad**.



Key Security Concepts

- **Confidentiality**

- Preserving authorized restrictions on information access and disclosure, including means for protecting personal privacy and proprietary information

- **Integrity**

- Guarding against improper information modification or destruction, including ensuring information nonrepudiation and authenticity

- **Availability**

- Ensuring timely and reliable access to and use of information

The Security Problem

- System secure if resources used and accessed as intended under all circumstances
 - Unachievable
- **Intruders** (crackers) attempt to breach security
- **Threat** is potential security violation
- **Attack** is attempt to breach security (which can be accidental or malicious)
- Easier to protect against accidental than malicious misuse

Security Violations

- Breach of confidentiality
 - Unauthorized reading of data
- Breach of integrity
 - Unauthorized modification of data
- Breach of availability
 - Unauthorized destruction of data
- Theft of service
 - Unauthorized use of resources
- Denial of service (DOS)
 - Prevention of legitimate use

Systems Security

- Computer Security Overview
- **Problems: Security Violation Methods**
- Actions to Mitigate the Problems
- Authentication, Passwords and Alternatives
- More on Threats

Some Computer Security Challenges (1)

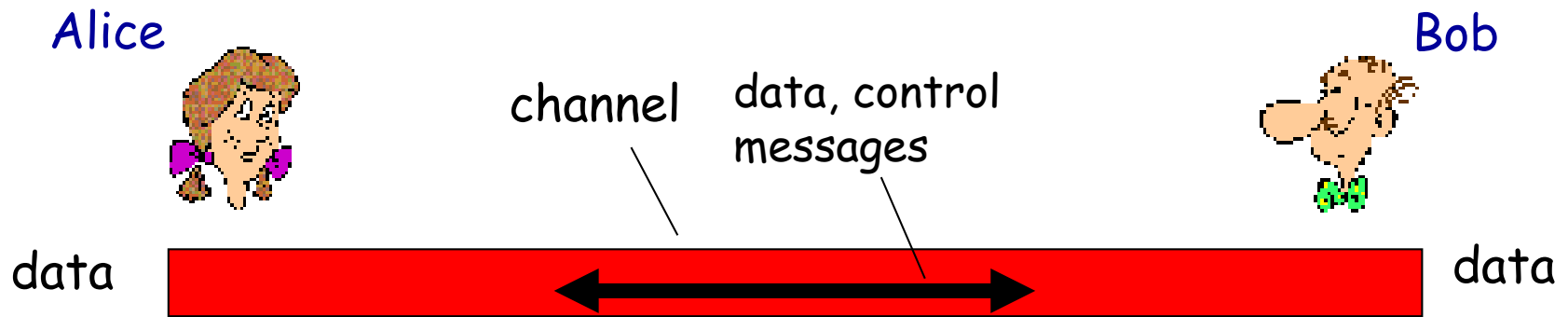
- Computer security is not as simple as it might first appear to the novice
- Potential **attacks on the security features** must be considered
- Procedures used to provide particular services are often counterintuitive
- **Physical and logical placement** needs to be determined
- **Additional algorithms** or protocols are usually needed on top of what may be provided by the functional requirements of the system

Some Computer Security Challenges (2)

- Attackers only need to find **a single weakness**, the developer needs to find **all weaknesses**
- Users and system managers tend to not see the benefits of security until a failure occurs
- Security requires **regular and constant monitoring**
- Security is often an **afterthought** incorporated into a system after the design is complete
- Thought of as an **impediment** to efficient and user-friendly operation

Friends and enemies: Alice, Bob, ...

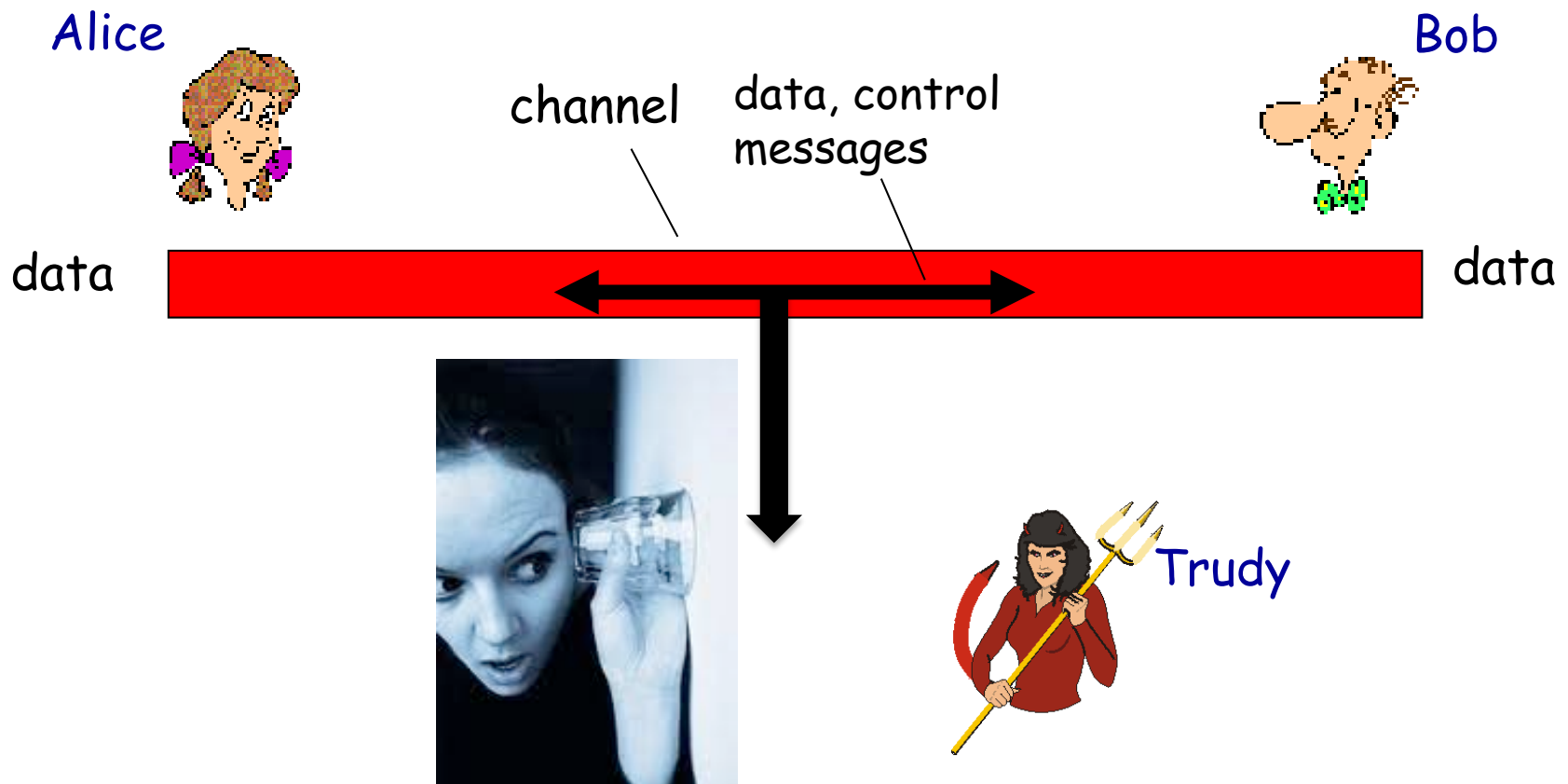
- Well-known in network security world after Rivest classic RSA paper (<https://dl.acm.org/doi/10.1145/359340.359342>).



- Carol*, *Dan* third / fourth participant
- Chuck*, a participant usually of malicious intent, *Craig*, cracker
- Eve*, an eavesdropper
- Mallory* or *Mallet*, a malicious attacker
- Trudy*, an intruder.
- Trent*, a trusted arbitrator, ...

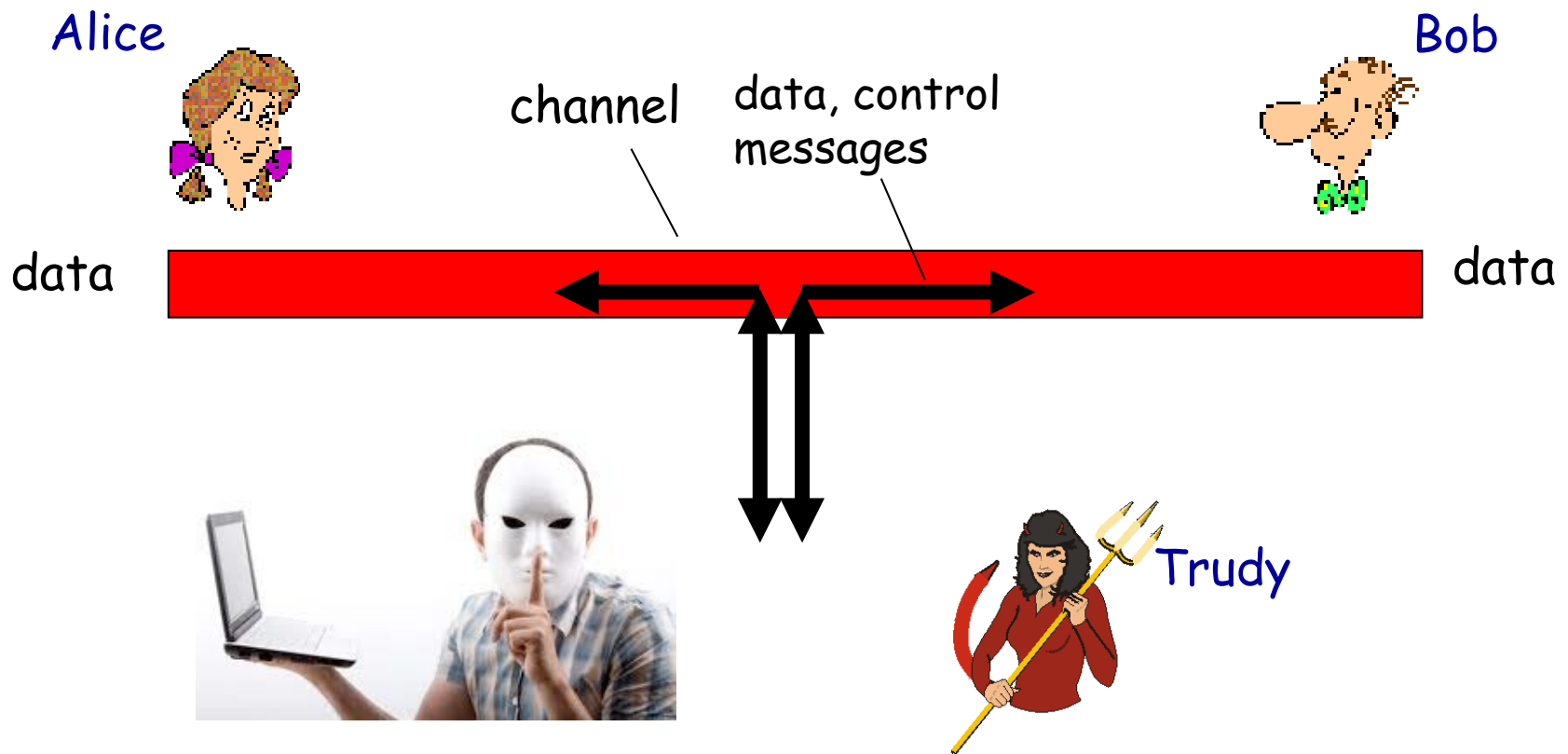
Potential problems for Bob and Alice (Security Violation Methods)

- Eavesdroppers in the conversation



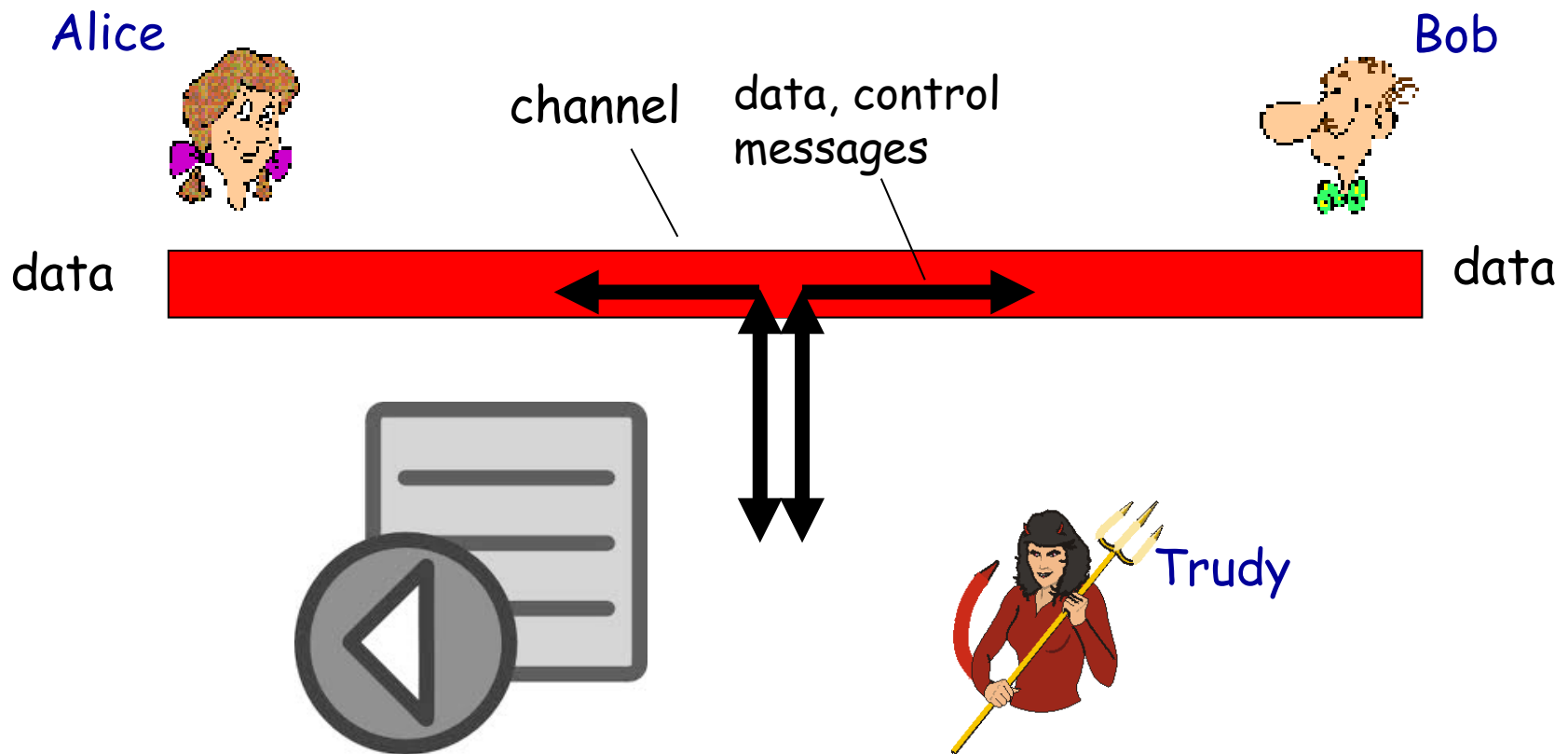
Potential problems for Bob and Alice (Security Violation Methods)

- Impersonation of users, Masquerading breach authentication



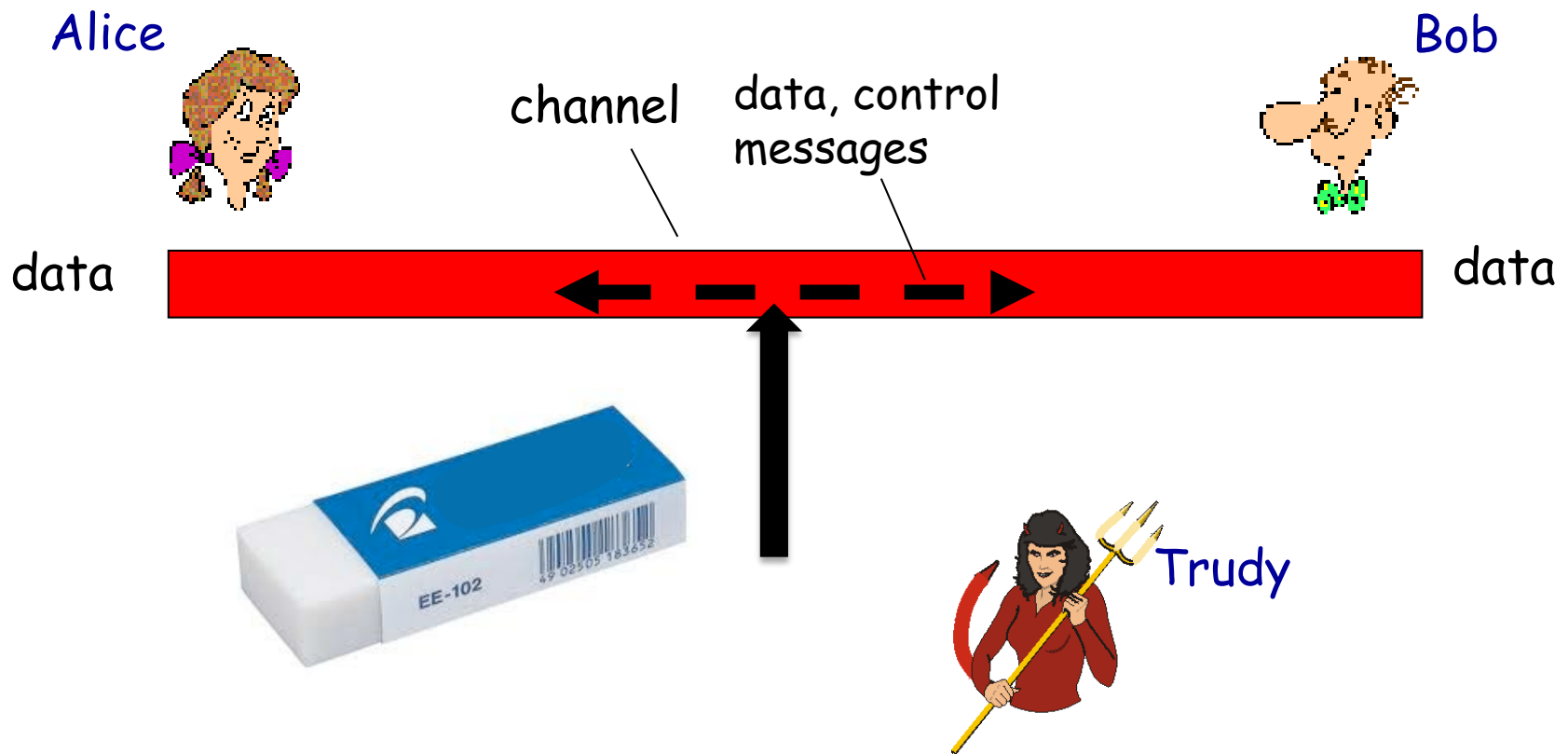
Potential problems for Bob and Alice (Security Violation Methods)

- Re-play of genuine / bogus messages



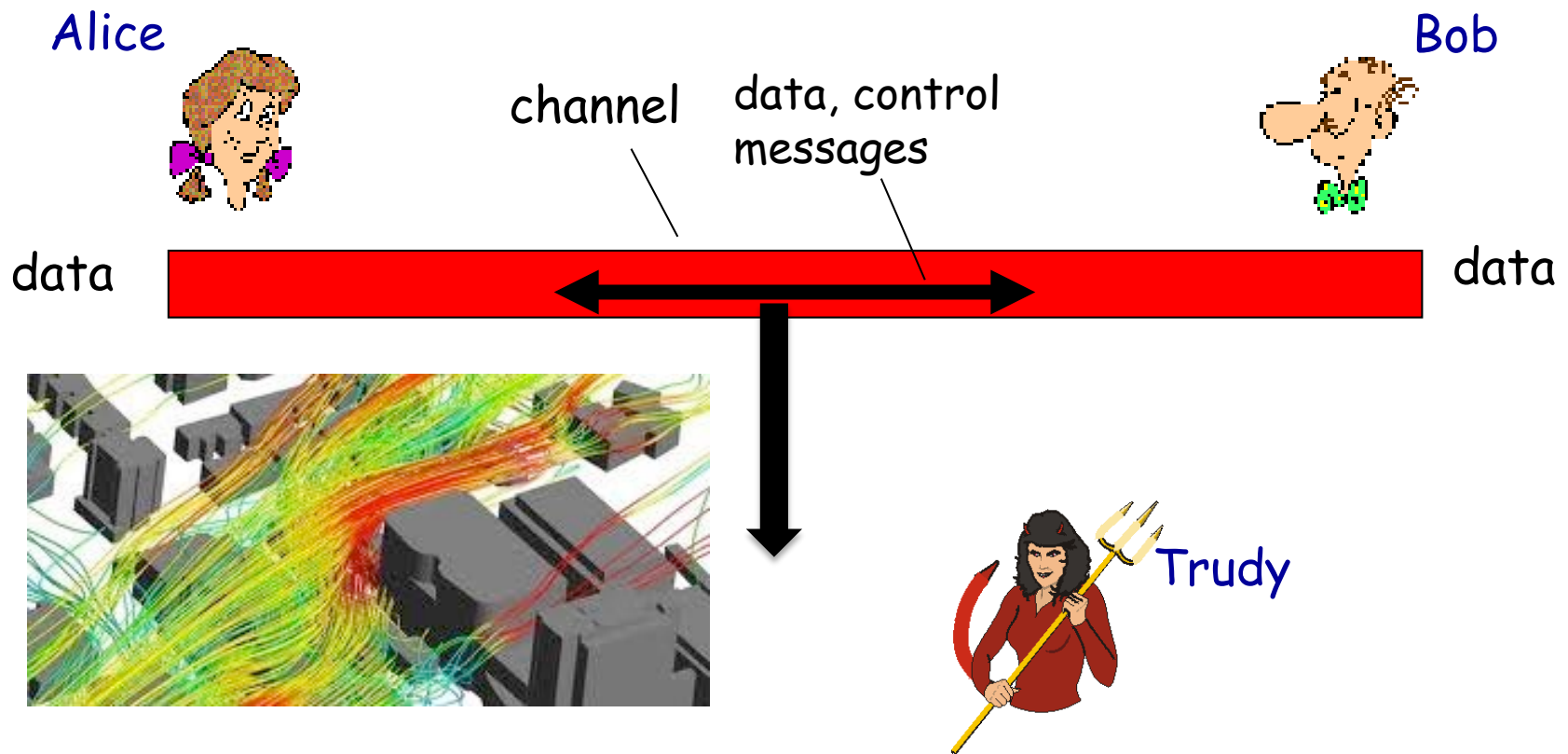
Potential problems for Bob and Alice (Security Violation Methods)

- Deleting or modifying messages



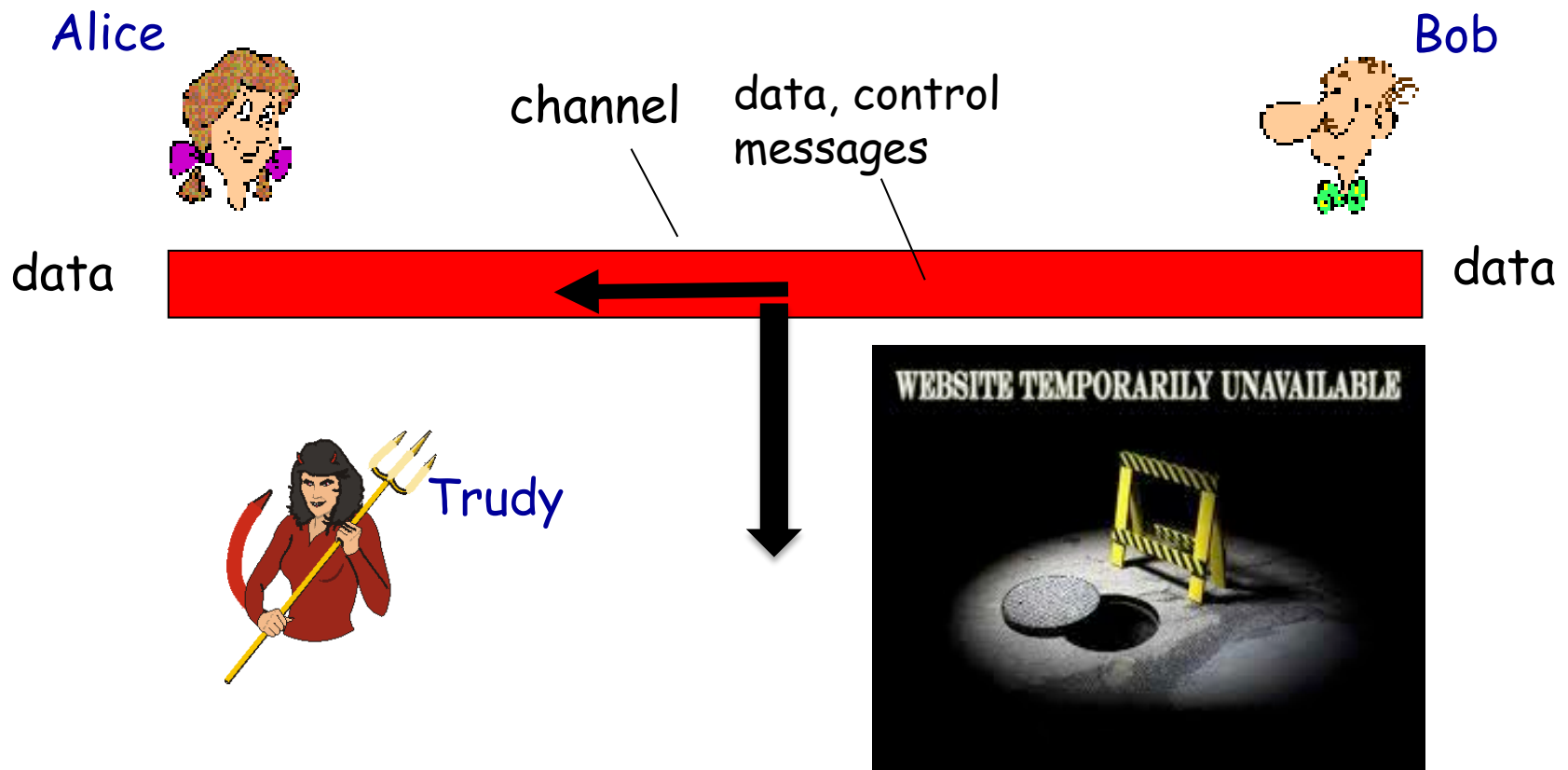
Potential problems for Bob and Alice (Security Violation Methods)

- Analysing the flow



Potential problems for Bob and Alice (Security Violation Methods)

- Blocking / diverting



Systems Security

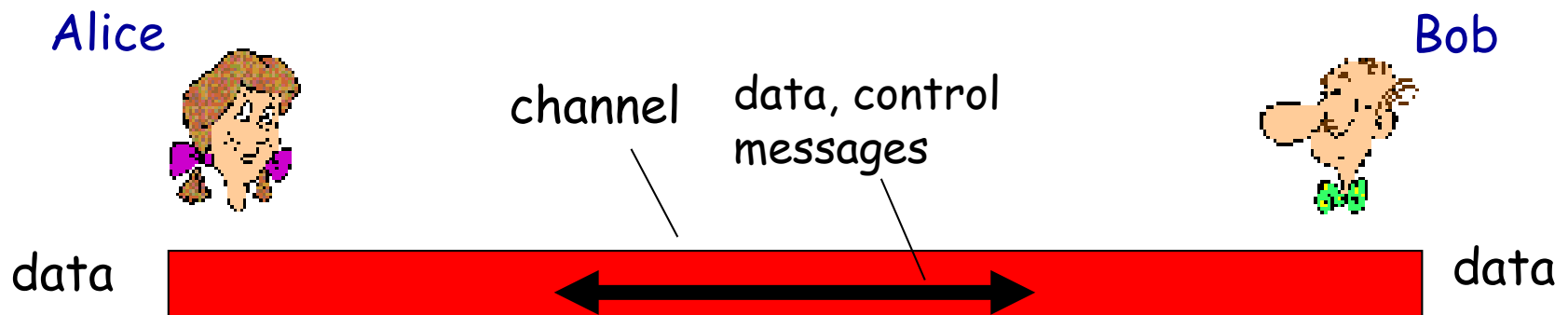
- Computer Security Overview
- Problems: Security Violation Methods
- **Actions to Mitigate the Problems**
- Authentication, Passwords and Alternatives
- More on Threats

Security Measure Levels

- Impossible to have absolute security, but make cost to perpetrator sufficiently high to deter most intruders
- Security must occur at four levels to be effective:
 - Physical
 - Data centers, servers, connected terminals
 - Human
 - Avoid social engineering, phishing, dumpster diving
 - Operating System
 - Protection mechanisms, debugging
 - Network
 - Intercepted communications, interruption, DOS
- Security is as weak as the weakest link in the chain
- But can too much security be a problem?

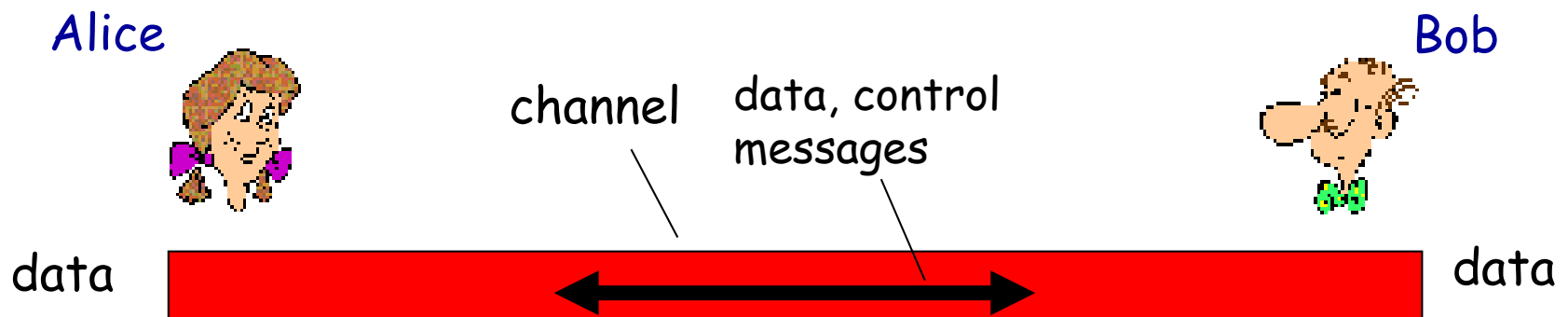
Potential Actions by Alice and Bob

- **Authentication:** sender, receiver want to confirm identity of each other
- RFC (request for comments) 2828 defines user authentication as:
 - *The process of verifying an identity claimed by or for a system entity.*



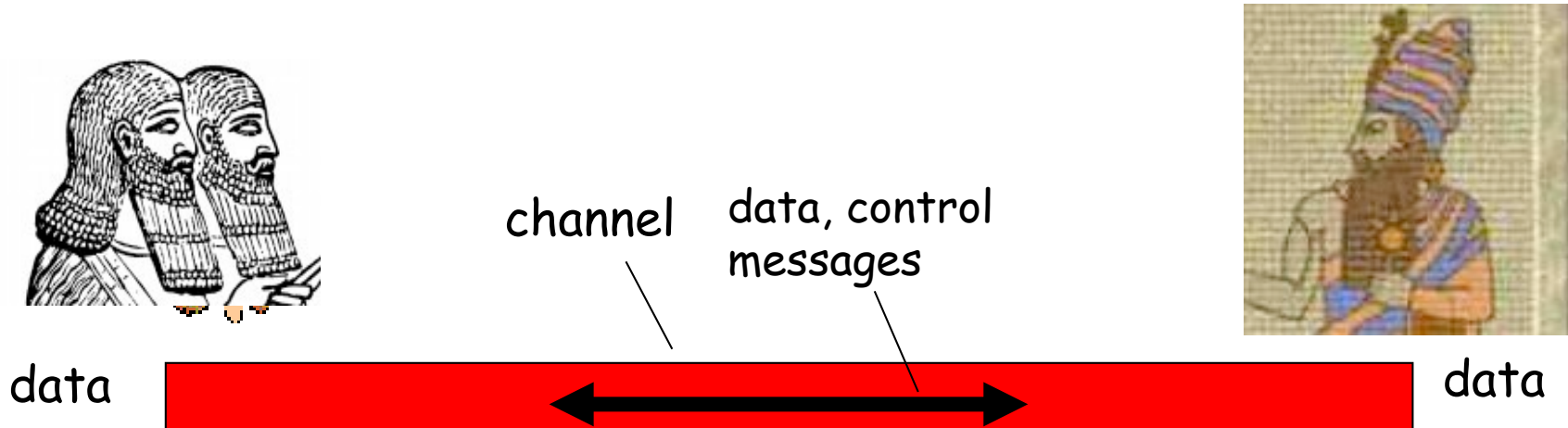
Potential Actions by Alice and Bob

- **Message integrity:** sender, receiver want to ensure message not altered (in transit, or afterwards) without detection.



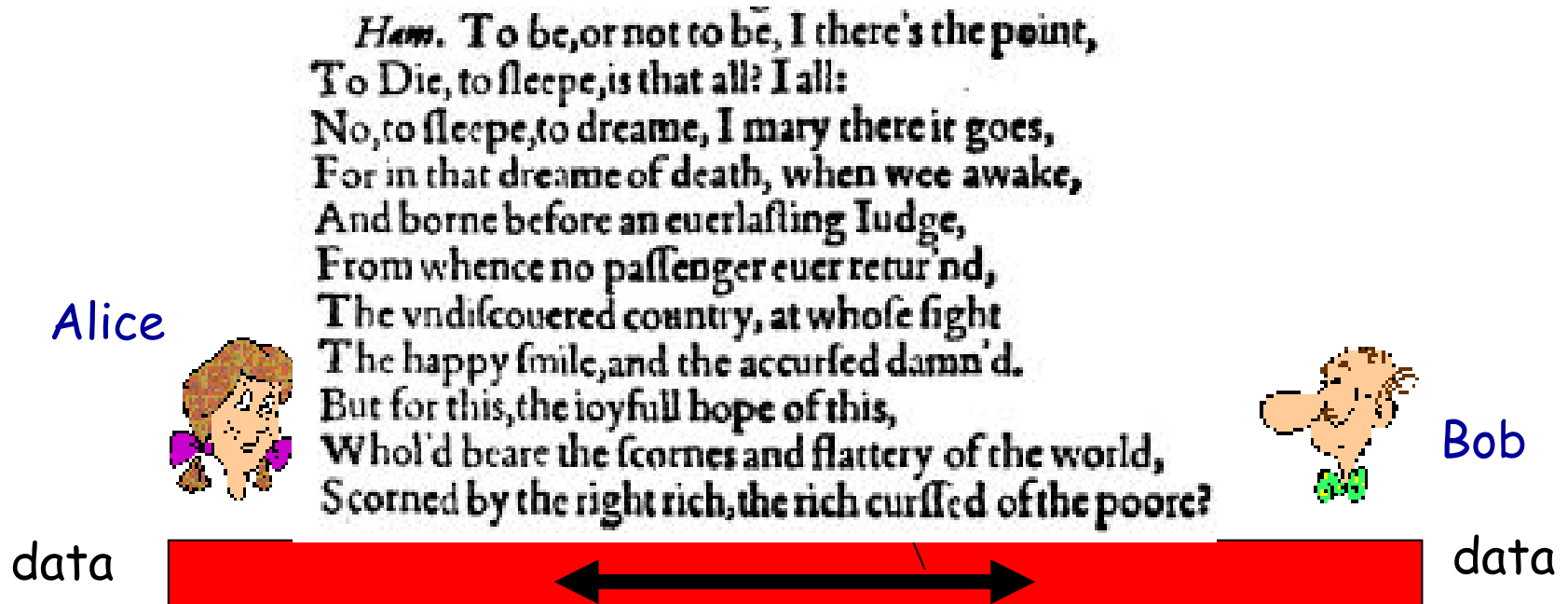
Potential Actions by Alice and Bob

- **Confidentiality:** only sender, intended receiver should “understand” message contents



Potential Actions by Alice and Bob

- **Stealth:** only sender and the intended receiver know that a hidden message is being sent



Systems Security

- Computer Security Overview
- Problems: Security Violation Methods
- Actions to Mitigate the Problems
- Authentication, Passwords and Alternatives
- More on Threats

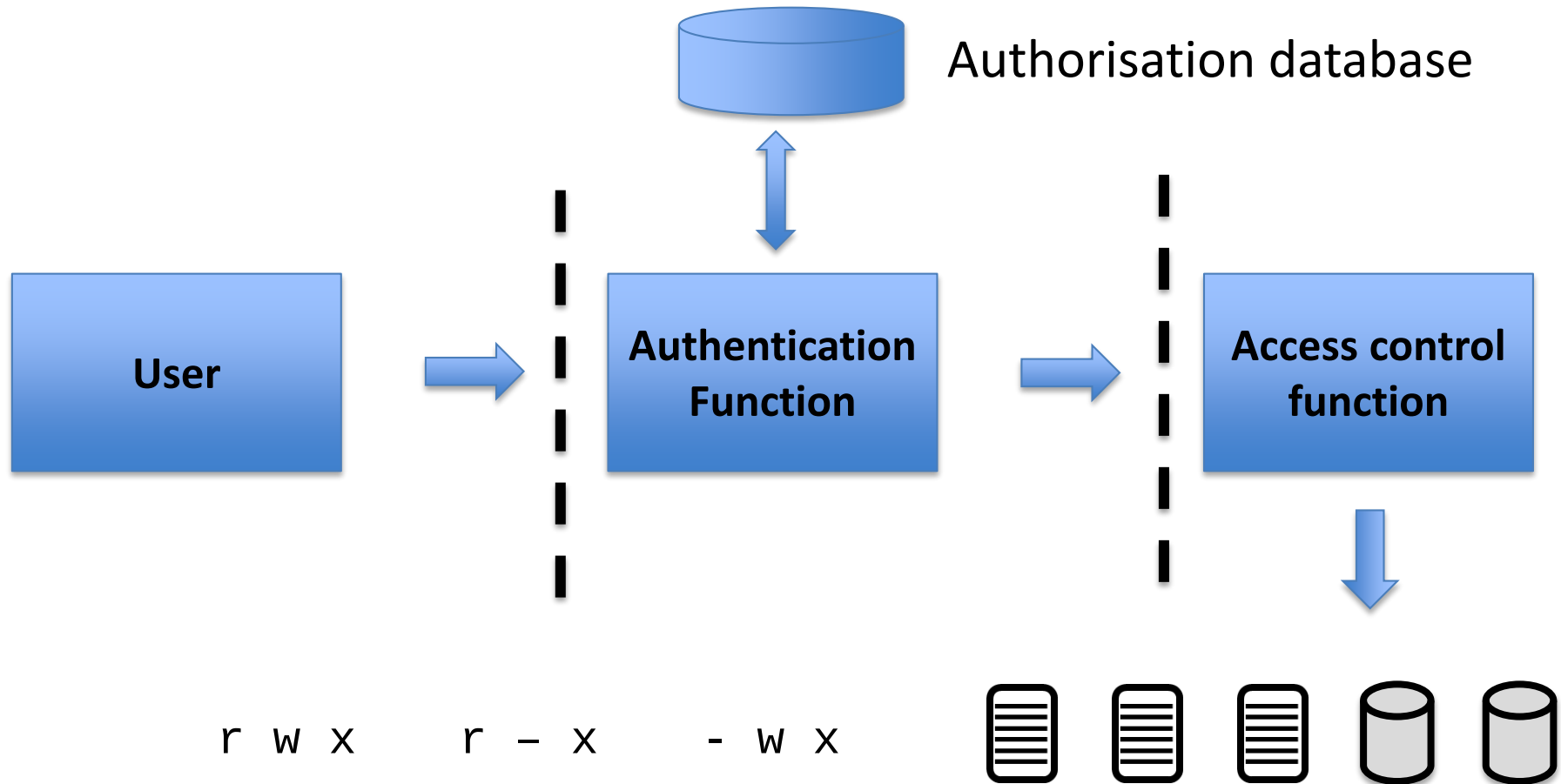
Authentication

- Authentication is a **fundamental** building block of computer security and a primary line of defence
- It provides the basis for access control and user accountability
- RFC (request for comments) 2828 defines user authentication as:
 - *The process of verifying an identity claimed by or for a system entity.*

Authentication Process

- Two broad steps in the authentication process:
 - **Identification** step presenting an identifier to the security system
 - **Verification** step presenting or generating authentication information that corroborates the binding between the entity and the identifier

Authentication Process



Means of Authentication

- Three main aspects under the user's control which can be used to authenticate a user:
 - The user's **possession** of something (a key or card)
 - The user's **knowledge** of something (a user identifier and password)
 - An **attribute** of the user (fingerprint, retina pattern, or signature)

Password Authentication

- Widely used line of defence against intruders
- User provides name/ID and password
- System compares password with the one stored for that specified ID
- Once authenticated, the user ID:
 - determines if the user is authorized to access the system at all
 - determines the user's privileges (access rights) if so

Password Vulnerabilities

- Offline dictionary attack
- Specific account attack
- Popular password attack
- Workstation hijacking
- Exploiting user mistakes
- Exploiting multiple password use
- Electronic monitoring



Countermeasures

- Controls to prevent unauthorized access to password file
- Intrusion detection measures
- Rapid reissuance of compromised passwords
- Account lockout mechanisms
- Policies to inhibit users from selecting common passwords
- Training in and enforcement of password policies
- Automatic workstation logout
- Policies against similar passwords on network devices

Password Selection Techniques

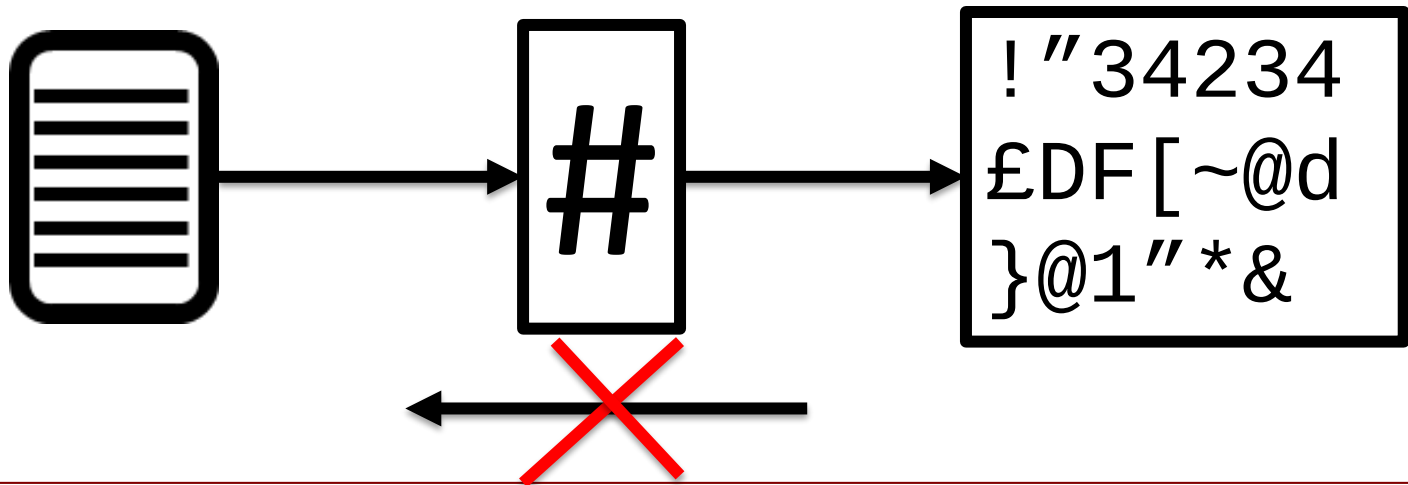
- **User education**
 - Users can be told the importance of using hard to guess passwords and can be provided with guidelines for selecting strong passwords
- **Computer generated passwords**
 - Users have trouble remembering them
- **Reactive password checking**
 - System periodically runs its own password cracker to find guessable passwords
- **Proactive password checking**
 - User is allowed to select their own password, however the system checks to see if the password is allowable, and if not, rejects it
 - Goal is to eliminate guessable passwords while allowing the user to select a password that is memorable

Proactive Password Checking

- Rule enforcement
 - Specific rules that passwords must adhere to
 - But if attackers know the rules they can optimize their attacks by only trying valid passwords
- Password "naughty list"
 - Compile a large dictionary of passwords not to use
 - Drawback: to be useful this must be very large
 - Occupies a big chunk of memory
 - Slow to search (slow login, waste of CPU cycles)

Hashed Passwords

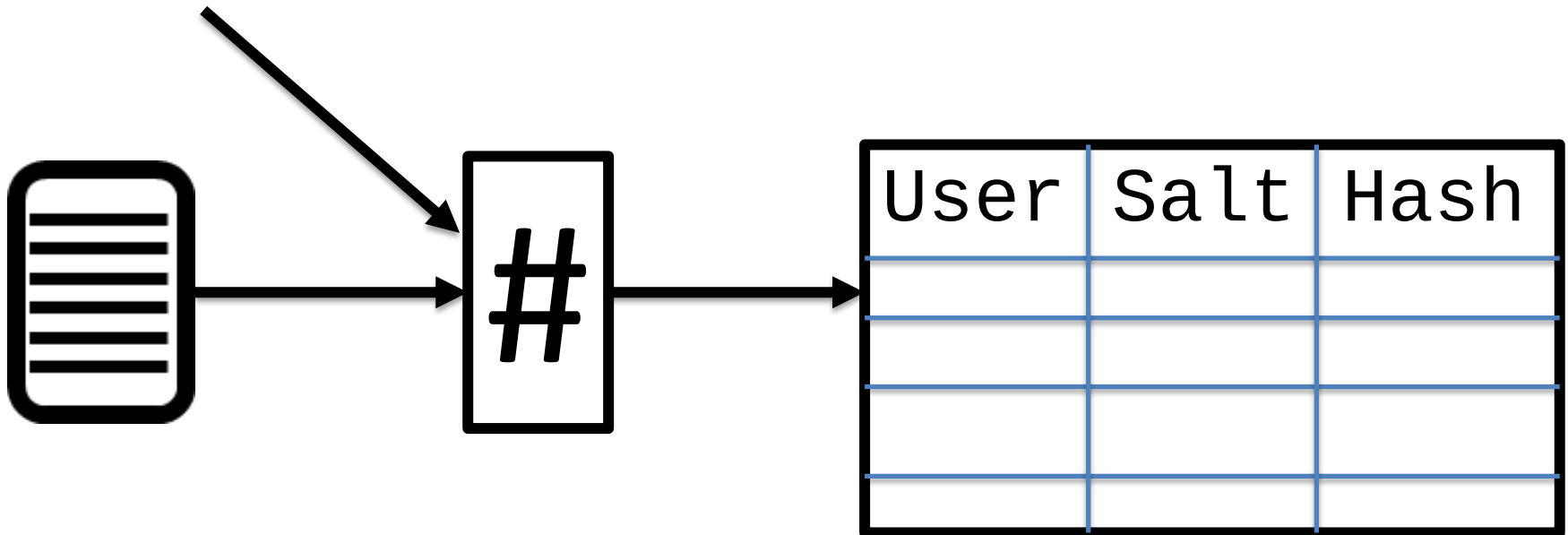
- A hash function will convert an input sequence (can be plain text of arbitrary length, the message) to another sequence, normally of fixed length (the hash)
- It should be a one-way function, that is, a function which is practically infeasible to invert



Hashed Passwords

- Extra elements: Salt and Slow

SALT



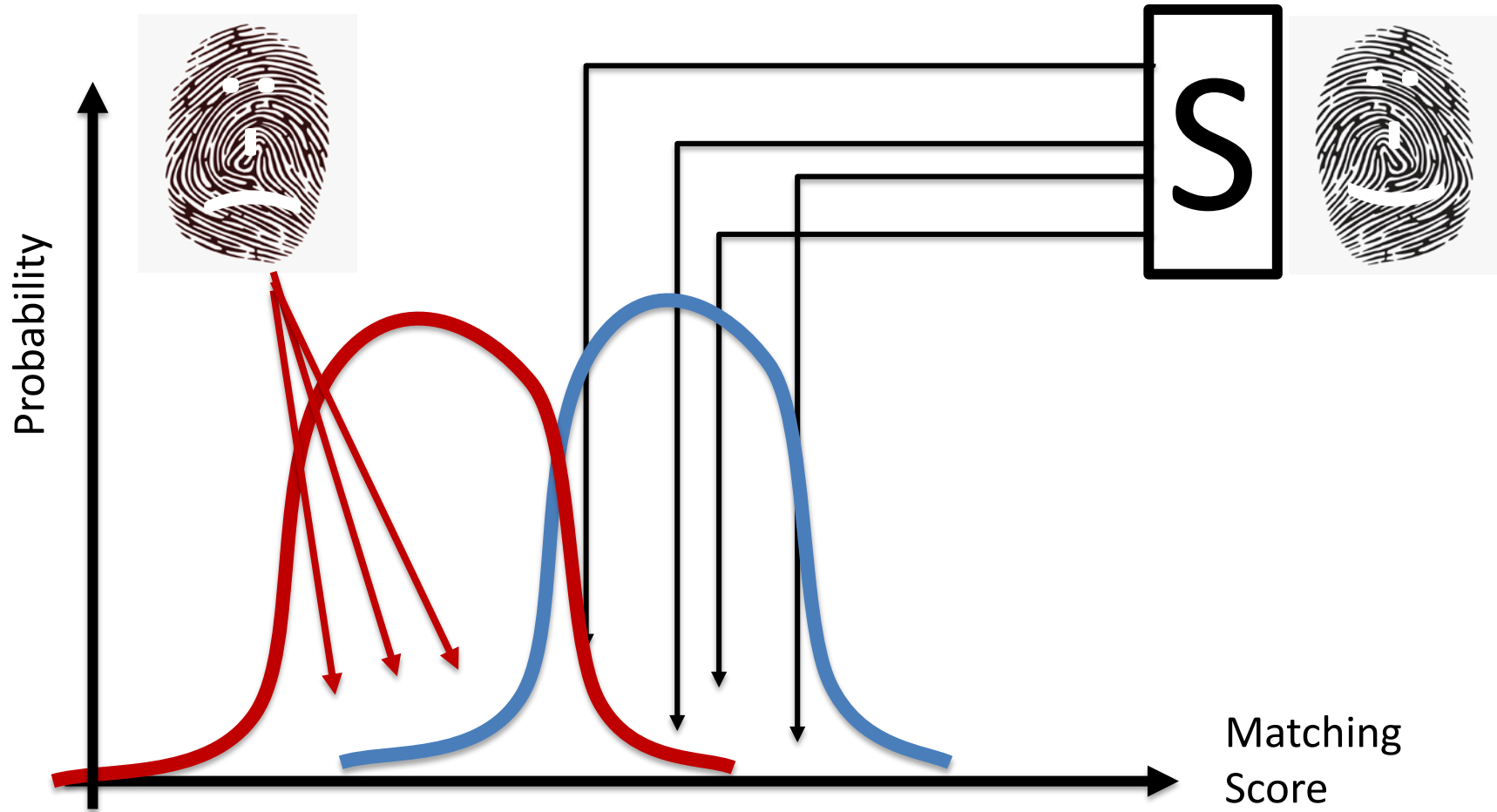
Password implementation

- **Unix original implementation**
- Eight ASCII characters
- Add 12 bit salt
- Encrypt 25 times (slow down)
- Translate to an 11 character sequence
- **Improved implementations**
- MD5 -> Blowfish block
- Salt 48 -> 128 bits
- Password length unlimited
- 128 -> 192 hash

Alternatives to passwords

- Attempts to authenticate an individual based on unique physical characteristics
- Based on pattern recognition
- Is technically complex and expensive when compared to passwords and tokens
- Physical characteristics used include:
 - Facial characteristics,
 - Fingerprints,
 - Hand geometry,
 - Retinal pattern,
 - Iris,
 - Signature,
 - Voice, ...

Issues of biometrics



Systems Security

- Computer Security Overview
- Problems: Security Violation Methods
- Actions to Mitigate the Problems
- Authentication, Passwords and Alternatives
- More on Threats

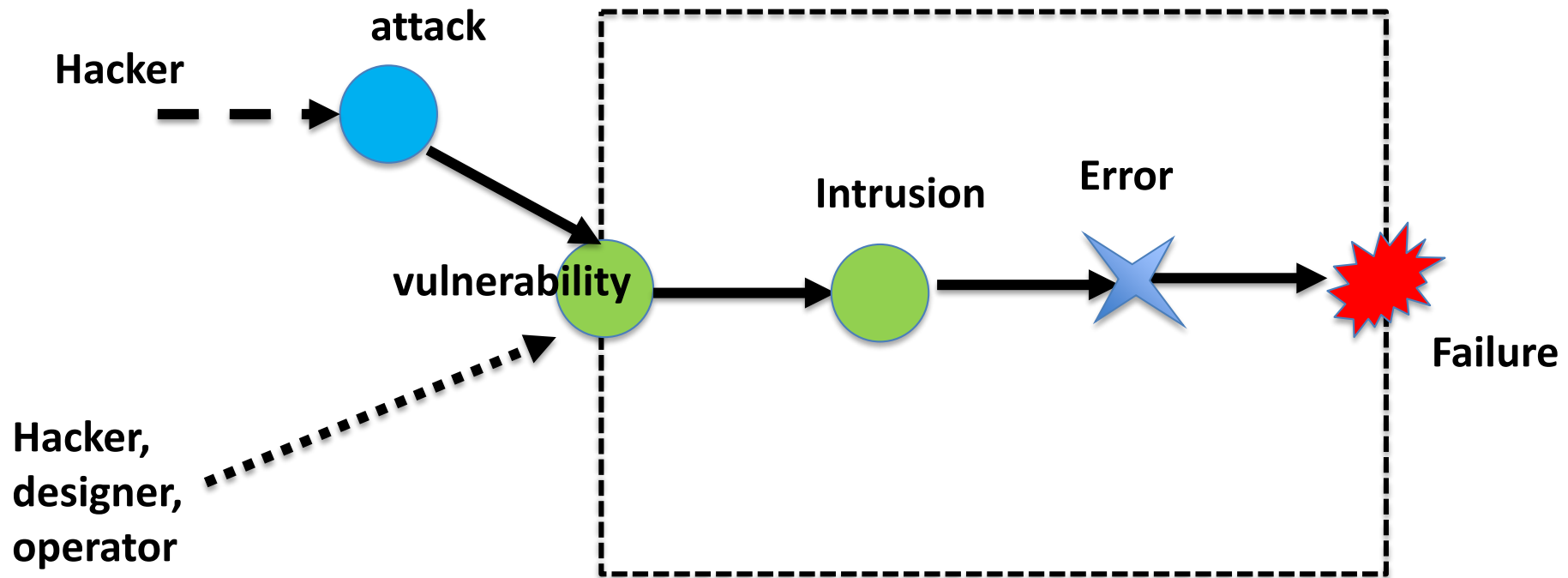
Threats

- Threats can be of various types and in different locations, to understand them better the following terminology is adopted from an EU project:
 - Malicious-and Accidental-Fault Tolerance for Internet Applications (MAFTIA)
 - <http://research.cs.ncl.ac.uk/cabernet/www.laas.research.ec.org/maftia/>
- The deliverable D21 of MAFTIA is recommended for further reading:
 - <http://research.cs.ncl.ac.uk/cabernet/www.laas.research.ec.org/maftia/deliverables/D21.pdf>

Terminology

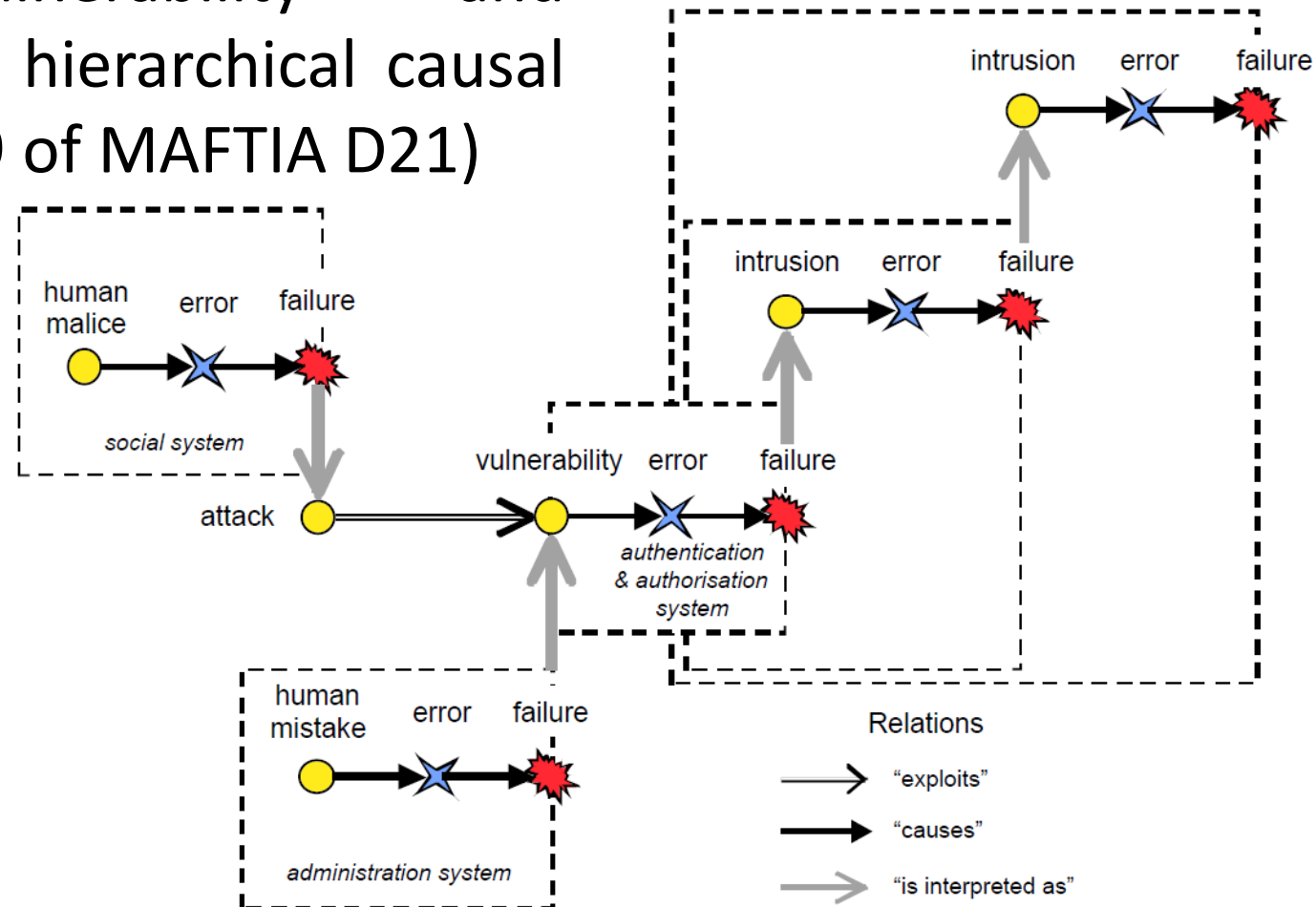
- **Fault:** adjudged or hypothesised cause of an error
- **Error:** that part of the system state that may lead to failure
- **Failure:** delivered service deviates from implementing the system function
- **Attack:** a malicious interaction *fault*, through which an attacker aims to deliberately violate one or more security properties; an *intrusion* attempt.
- **Vulnerability:** a fault created during development of the system, or during operation, that could be exploited to create an *intrusion*
- **Intrusion:** a *malicious*, externally-induced *fault* resulting from an *attack* that has been successful in exploiting a *vulnerability*

Terminology



Hierarchical causal chain

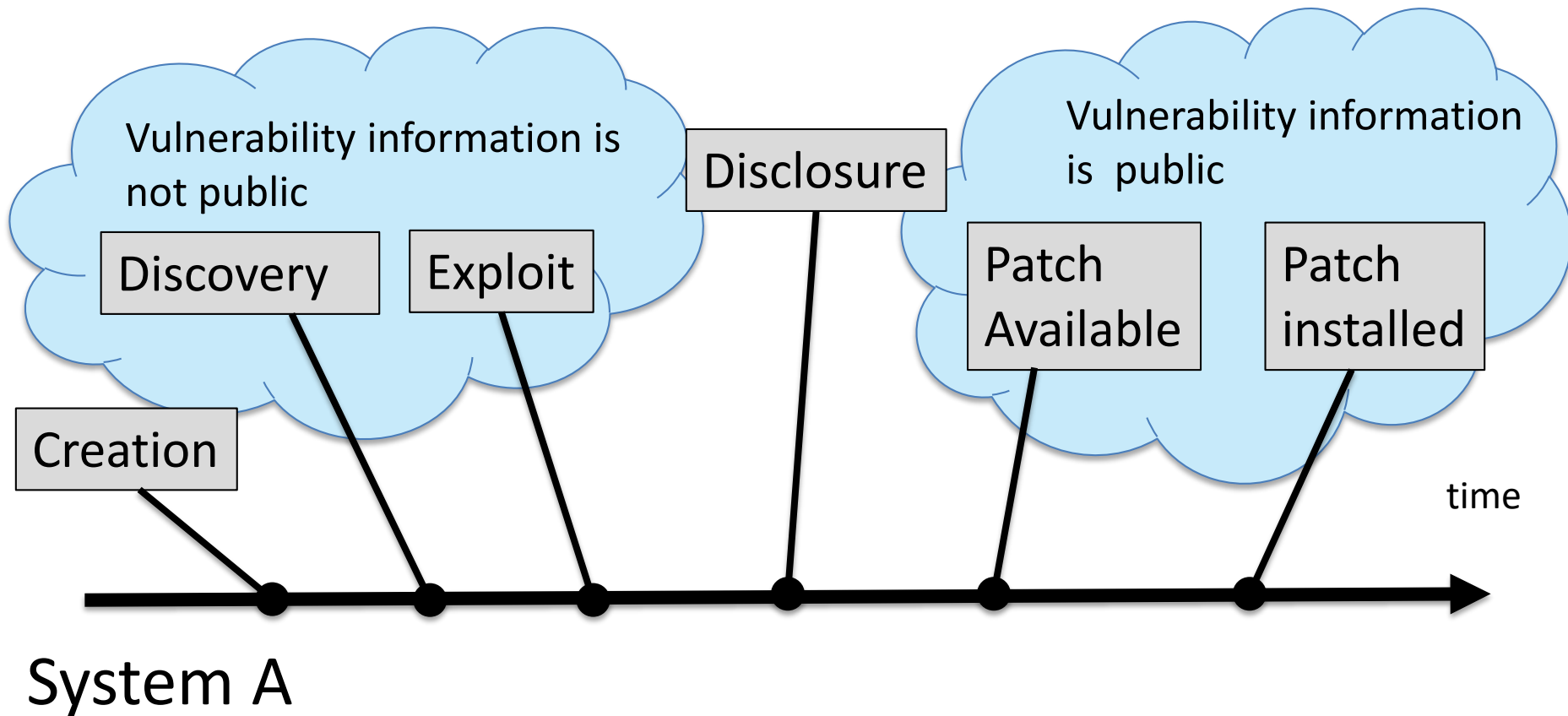
- Attack, vulnerability and intrusion in a hierarchical causal chain (figure 9 of MAFTIA D21)



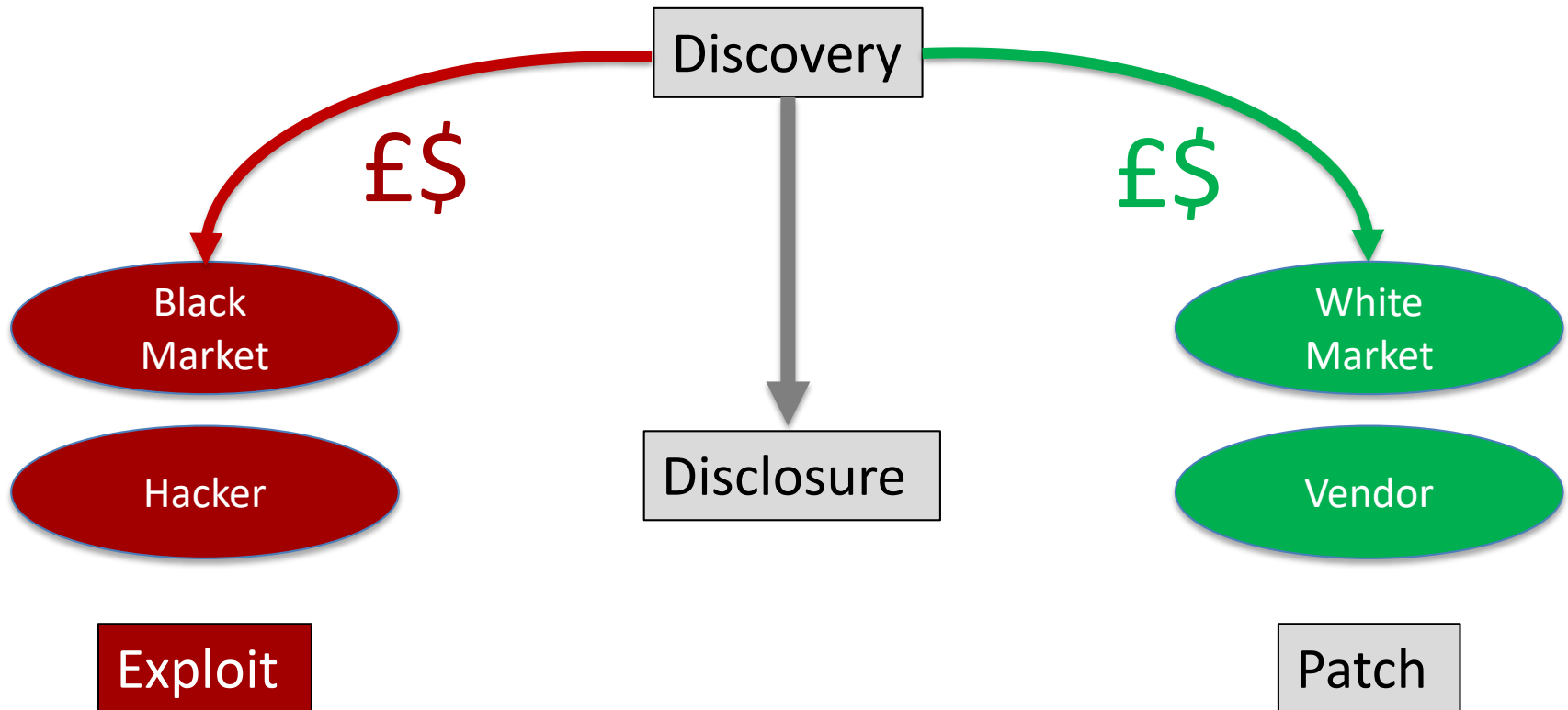
Vulnerability lifecycle

- Another useful way of conceptualising the security threats is to look at the lifecycle of a vulnerability
- A very good resource for this is the PhD thesis of Stefan Frei, from IBM Zurich. A shorter version of this work is given in the following publication (<http://weis09.infoseccon.net/files/103/paper103.pdf>)
- Looking at the vulnerability lifecycle in this way gives us insight on the different types and stages of risks that a system may have
- It also gives us insight on the economic aspects of vulnerability disclosure and the different types of incentives that effects the decision making of both attackers and defenders

Vulnerability lifecycle



Economics of a vulnerability



Historical vulnerabilities

- <https://www.cvedetails.com/top-50-products.php?year=2003>

Top 50 Products By Total Number Of "Distinct" Vulnerabilities in 2000

Go to year: [1999](#) [2000](#) [2001](#) [2002](#) [2003](#) [2004](#) [2005](#) [2006](#) [2007](#) [2008](#) [2009](#) [2010](#) [2011](#) [2012](#) [2013](#) [2014](#) [2015](#) [2016](#) [2017](#) [2018](#) [2019](#) [2020](#) [2021](#) [2022](#) [2023](#) [2024](#) [2025](#) [2026](#) [2027](#) [2028](#) [2029](#) [2030](#) [2031](#) [2032](#) [2033](#) [2034](#) [2035](#) [2036](#) [2037](#) [2038](#) [2039](#) [2040](#) [2041](#) [2042](#) [2043](#) [2044](#) [2045](#) [2046](#) [2047](#) [2048](#) [2049](#) [2050](#) [2051](#) [2052](#) [2053](#) [2054](#) [2055](#) [2056](#) [2057](#) [2058](#) [2059](#) [2060](#) [2061](#) [2062](#) [2063](#) [2064](#) [2065](#) [2066](#) [2067](#) [2068](#) [2069](#) [2070](#) [2071](#) [2072](#) [2073](#) [2074](#) [2075](#) [2076](#) [2077](#) [2078](#) [2079](#) [2080](#) [2081](#) [2082](#) [2083](#) [2084](#) [2085](#) [2086](#) [2087](#) [2088](#) [2089](#) [2090](#) [2091](#) [2092](#) [2093](#) [2094](#) [2095](#) [2096](#) [2097](#) [2098](#) [2099](#)

Product Name		Go to year: 1999 2000	
1	Linux		
2	Windows 2000	1	Mac Os X
3	Windows Nt	2	Firefox
4	Freebsd	3	IE
5	Internet Information	4	Linux Kernel
6	Hp-ux	5	Seamonkey
7	IE	6	Thunderbird
8	Mandrake Linux	7	Database Server
9	Suse Linux	8	Windows Xp
10	Internet Information	9	Mac Os X Server
11	Debian Linux	10	Windows 2003 Se
		11	Solaris

Top 50 Products By Total Number Of "Distinct" Vulnerabilities in 2006

[9](#) [2020](#) [2021](#) [All Time Leaders](#)

Go to year: 1999 2000	
	Product Name
1	Mac Os X
2	Firefox
3	IE
4	Linux Kernel
5	Seamonkey
6	Thunderbird
7	Database Server
8	Windows Xp
9	Mac Os X Server
10	Windows 2003 Se
11	Solaris

Top 50 Products By Total Number Of "Distinct" Vulnerabilities in 2015

Go to year: [1999](#) [2000](#) [2001](#)

	Prod
1	<u>Mac Os X</u>
2	<u>Iphone Os</u>
3	<u>Flash Player</u>
4	<u>Opensuse</u>
5	<u>Ubuntu Linux</u>
6	<u>Air Sdk</u>
7	<u>AIR</u>
8	<u>Air Sdk & Compiler</u>
9	<u>Debian Linux</u>
10	<u>Internet Explorer</u>
11	<u>Chrome</u>

Top 50 Products By Total Number Of "Distinct" Vulnerabilities in 2019

Go to year: [1999](#) [2000](#) [2001](#) [2002](#) [2003](#) [2004](#) [2005](#) [2006](#) [2007](#) [2008](#) [2009](#) [2010](#) [2011](#) [2012](#) [20](#)

	Product Name	Vendor Name	Product Type	Number of Vulnerabilities
1	Android	Google	OS	414
2	Debian Linux	Debian	OS	360
3	Windows Server 2016	Microsoft	OS	357
4	Windows 10	Microsoft	OS	357
5	Windows Server 2019	Microsoft	OS	351
6	Acrobat Reader Dc	Adobe	Application	342
7	Acrobat Dc	Adobe	Application	342
8	Cpanel	Cpanel	Application	321
9	Windows 7	Microsoft	OS	250
10	Windows Server 2008	Microsoft	OS	248
11	Windows Server 2012	Microsoft	OS	246

Historical vulnerabilities

- <https://www.cvedetails.com/top-50-vendors.php>
- <https://www.cvedetails.com/top-50-vendor-cvssscore-distribution.php>

Top 50 Vendors By Total Number Of "Distinct" Vulnerabilities

Go to year: [1999](#) [2000](#) [2001](#) [2002](#) [2003](#) [2004](#) [2005](#) [2006](#) [2007](#) [2008](#) [2009](#) [2010](#) [2011](#) [2012](#) [2013](#) [2014](#)

Vendor Name	Number of Products	Number of Vulnerabilities	#Vulnerabilities/#Products
1 Microsoft	529	6814	13
2 Oracle	644	6115	9
3 IBM	1064	4679	4
4 Google	84	4572	54
5 Apple	119	4512	38
6 Cisco	3676	4167	1
7 Adobe	132	3314	25
8 Debian	97	3197	33
9 Redhat	301	2805	9
10 Linux	17	2370	139
11 Mozilla	24	2199	92
12 Canonical	30	2025	68
13 HP	3579	1794	1
14 SUN	204	1628	8
15 Opensuse	25	1315	53
16 Apache	198	1218	6
17 Fedoraproject	17	757	45
18 GNU	101	738	7
19 Novell	118	665	6
20 PHP	22	626	28
21 Qualcomm	299	602	2
22 Wireshark	1	576	576
23 SAP	226	565	3
24 Imagemagick	3	546	182
25 Huawei	1177	536	0

CVSS Score Distribution For Top 50 Vendors By Total Number Of "Distinct" Vulnerabilities

	Vendor Name	Number of Total Vulnerabilities	# Of Vulnerabilities										Weighted Average
			0-1	1-2	2-3	3-4	4-5	5-6	6-7	7-8	8-9	9+	
1	Microsoft	6814	2	103	388	111	1016	805	389	1730	30	2240	7.50
2	Oracle	6115	2	115	272	463	1798	1670	681	518	25	571	6.00
3	IBM	4679	2	65	295	784	1263	829	460	563	30	388	5.80
4	Google	4572		23	119	18	1057	514	651	1084	31	1075	7.20
5	Apple	4512	1	53	272	44	781	544	1128	700	15	969	7.00
6	Cisco	4167	1	5	69	111	816	933	569	1169	44	450	6.90
7	Adobe	3314			18	3	409	328	176	180	1	2199	8.70
8	Debian	3197		31	130	76	883	618	620	684	7	148	6.30
9	Redhat	2805		54	212	117	693	525	437	542	9	216	6.20
10	Linux	2370	1	93	346	54	738	145	186	665	7	135	5.90
11	Mozilla	2199		9	79	12	438	453	262	388	1	557	7.20
12	Canonical	2025		30	108	58	610	387	334	393	4	101	6.20
13	HP	1794	1	11	63	43	288	250	148	406	26	558	7.50
14	SUN	1628	3	26	105	45	311	283	119	421	4	311	6.80
15	Opensuse	1315		19	66	56	291	296	253	209	3	122	6.40
16	Apache	1218		7	40	31	320	399	138	217	2	64	6.20
17	Fedoraproject	757		8	38	25	185	204	118	150	1	28	6.20
18	GNU	738	1	12	45	29	196	167	126	128		34	6.10
19	Novell	665	1	7	27	9	152	145	46	126		152	6.90
20	PHP	626		1	21	6	68	190	88	210	1	41	6.80
21	Qualcomm	602			19	3	72	70	19	164	10	245	8.00
22	Wireshark	576			24	32	184	261	7	46	3	19	5.80
23	SAP	565		3	13	26	141	171	67	98	1	45	6.40
24	Imagemagick	546			2	1	297	45	106	87		8	6.00
25	Huawei	536		3	46	14	127	106	67	113	10	50	6.40

Program Threats

- Many variations, many names
- Trojan Horse
 - Code segment that misuses its environment
 - Exploits mechanisms for allowing programs written by users to be executed by other users
 - Spyware, pop-up browser windows, covert channels
 - Up to 80% of spam delivered by spyware-infected systems
- Trap Door
 - Specific user identifier or password that circumvents normal security procedures
 - Could be included in a compiler
 - How to detect them?

Program Threats (Cont.)

- Logic Bomb
 - Program that initiates a security incident under certain circumstances
- Stack and Buffer Overflow
 - Exploits a bug in a program (overflow either the stack or memory buffers)
 - Failure to check bounds on inputs, arguments
 - Write past arguments on the stack into the return address on stack
 - When routine returns from call, returns to hacked address
 - Pointed to code loaded onto stack that executes malicious code
 - Unauthorized user or privilege escalation

Program Threats (Cont.)

- Viruses
 - Code fragment embedded in legitimate program
 - Self-replicating, designed to infect other computers
 - Very specific to CPU architecture, operating system, applications
 - Usually borne via email or as a macro

Program Threats (Cont.)

- Virus dropper inserts virus onto the system
- Many categories of viruses, literally many thousands of viruses
 - File / parasitic
 - Boot / memory
 - Macro
 - Source code
 - Polymorphic to avoid having a virus signature
 - Encrypted
 - Stealth
 - Tunneling

System and Network Threats (Cont.)

- Port scanning
 - Automated attempt to connect to a range of ports on one or a range of IP addresses
 - Detection of answering service protocol
 - Detection of OS and version running on system
 - nmap scans all ports in a given IP range for a response
 - nessus has a database of protocols and bugs (and exploits) to apply against a system
 - Frequently launched from zombie systems
 - To decrease trace-ability

System and Network Threats (Cont.)

- Denial of Service
 - Overload the targeted computer preventing it from doing any useful work
 - Distributed denial-of-service (DDOS) come from multiple sites at once
 - Consider the start of the IP-connection handshake (SYN)
 - How many started-connections can the OS handle?
 - Consider traffic to a web site
 - How can you tell the difference between being a target and being really popular?
 - Accidental – CS students writing bad `fork()` code
 - Purposeful – extortion, punishment

Countermeasure

- Involves four complementary courses of action:
- **Prevention** E.g. Secure encryption algorithms / Prevent unauthorized access to encryption keys
- **Detection** E.g. Intrusion detection systems / Detection of denial of service attacks
- **Recovery** E.g. Use of backup systems
- **Response** E.g. Upon detection, being able to halt an attack and prevent further damage

More information about security threats, vulnerabilities

- <https://us-cert.cisa.gov/>
- <https://www.ncsc.gov.uk/>
- <https://www.cvedetails.com/>
- <https://www.ibm.com/security/xforce>
- <https://www.broadcom.com/products/cyber-security> (Symantec)
- <https://www.broadcom.com/support/security-center/publications/archive>
- <http://cve.mitre.org/>

More information about security threats, vulnerabilities

- Buffer overflows
 - <http://www.youtube.com/watch?v=8xonDJe3Yxl>
- Hire the hackers:
 - http://www.ted.com/talks/misha_glenny_hire_the_hackers.html
- Check out other talks in the series “Hire the hackers”:
 - http://www.ted.com/playlists/10/who_are_the_hackers.html

Systems Security

- Computer Security Overview
- Problems: Security Violation Methods
- Actions to Mitigate the Problems
- Authentication, Passwords and Alternatives
- More on Threats